LinearSkills Academy

Class 11 — Dictionaries

Zero to Advance

Part 1 — The Basics

1. What is a Dictionary?

A dictionary stores data in key-value pairs, using curly brackets { }. Instead of finding items by position (like a list), you find them by name (the key).

```
student = {
    "name": "Ali",
    "age": 20,
    "city": "Karachi"
}
```

2. Accessing Values

```
print(student["name"])   # Ali
print(student["age"])    # 20
```

If the key does not exist, this way gives an error. A safer way is the .get() method (covered in the methods table below).

3. Adding and Updating Values

```
student["grade"] = "A"      # adds a new key-value pair
student["age"] = 21         # updates an existing value
print(student)
```

4. Removing Values

```
del student["city"]     # removes the key "city"
print(student)
```

Part 2 — Dictionary Methods

5. Common Methods

Method	Example	Description
`.get()`	`student.get("name")`	Gets a value safely (no error if key is missing)
`.keys()`	`student.keys()`	Gives all the keys
`.values()`	`student.values()`	Gives all the values
`.items()`	`student.items()`	Gives all key-value pairs together
`.pop()`	`student.pop("age")`	Removes a key and returns its value
`.update()`	`student.update({...})`	Adds or updates multiple pairs at once

`.clear()`	`student.clear()`	Removes everything from the dictionary

```
print(student.get("name"))          # Ali
print(student.get("email", "N/A"))  # N/A -- default value if key missing

print(student.keys())     # dict_keys(['name', 'age', 'grade'])
print(student.values())   # dict_values(['Ali', 21, 'A'])
print(student.items())    # dict_items([('name','Ali'), ('age',21), ('grade','A')])
```

Part 3 — Looping & Nested Dictionaries

6. Looping Through a Dictionary

```
student = {"name": "Ali", "age": 21, "grade": "A"}

# Loop through keys only
for key in student:
    print(key)

# Loop through keys and values together
for key, value in student.items():
    print(key, "->", value)
```

7. Nested Dictionaries

A dictionary can hold another dictionary inside it — useful for storing records with multiple fields.

```
students = {
    "s1": {"name": "Ali", "age": 20},
    "s2": {"name": "Sara", "age": 22}
}

print(students["s1"]["name"])   # Ali
print(students["s2"]["age"])    # 22
```

8. Looping Through Nested Dictionaries

```
for student_id, info in students.items():
    print(student_id, "->", info["name"], info["age"])
```

Part 4 — Going Further

9. Dictionary Comprehension

Just like list comprehension, but it builds a dictionary in one line.

```
squares = {i: i * i for i in range(5)}
print(squares)   # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

10. Checking if a Key Exists

```
student = {"name": "Ali", "age": 21}

if "name" in student:
    print("Key exists")

if "email" not in student:
```

```
    print("Key does not exist")
```

11. setdefault() — Get or Create in One Step

This gets a value if the key exists; if not, it creates the key with a default value you provide.

```
student = {"name": "Ali"}

age = student.setdefault("age", 18)
print(age)        # 18 (added because "age" was missing)
print(student)     # {'name': 'Ali', 'age': 18}
```

12. Merging Dictionaries

```
a = {"x": 1, "y": 2}
b = {"y": 5, "z": 3}

merged = {**a, **b}
print(merged)   # {'x': 1, 'y': 5, 'z': 3}  -- b overwrites shared keys
```

13. Sorting a Dictionary by Value

Dictionaries do not sort themselves, but you can build a sorted list of their items using sorted().

```
marks = {"Ali": 85, "Sara": 92, "Bilal": 78}

sorted_marks = sorted(marks.items(), key=lambda pair: pair[1], reverse=True)
print(sorted_marks)   # [('Sara', 92), ('Ali', 85), ('Bilal', 78)]
```

14. List vs Tuple vs Set vs Dictionary — Full Picture

Data Type	Use When...
List	Ordered collection that can change
Tuple	Fixed data that should never change
Set	Unique values only, order does not matter
Dictionary	You need to look up values by a name (key), not by position

15. Mini Demo

```
students = {
    "Ali": 85,
    "Sara": 92,
    "Bilal": 78
}

for name, marks in students.items():
    if marks >= 80:
        print(name, "- Pass with Distinction")
    else:
        print(name, "- Pass")
```